

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Mon 30 AUGUST 2021 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

SIGNATURE:

(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.00m

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R below, over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$R = a (b \mid c^+ a)^*$$

Answer the following questions:

- (a) Write all the strings x of the language of R that have a length less than or equal to four, i.e., $x \in L(R)$ with $|x| \leq 4$, in lexicographic order (with $a < b < c$).
 - (b) By means of the Berry-Sethi (BS) method, find a deterministic automaton A equivalent to the regular expression R .
 - (c) Is the deterministic automaton A found before minimal ? Justify your answer.
 - (d) By means of the Brzowski method (node elimination), starting from the automaton A found before, obtain a regular expression R' equivalent to A .
 - (e) (optional) Is the language $L(R)$ locally testable ? Formally prove your answer.
-

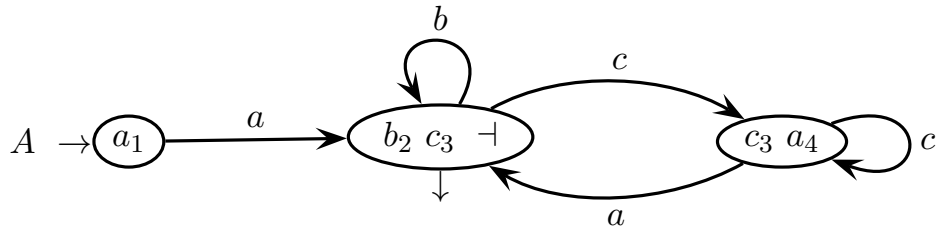
Solution

(a) Strings: $a, ab, ab^2, aca, ab^3, abca, acab$ and ac^2a

(b) Berry-Sethi:

$$R_{\#} = a_1 \left(b_2 \mid c_3^+ a_4 \right)^* \neg$$

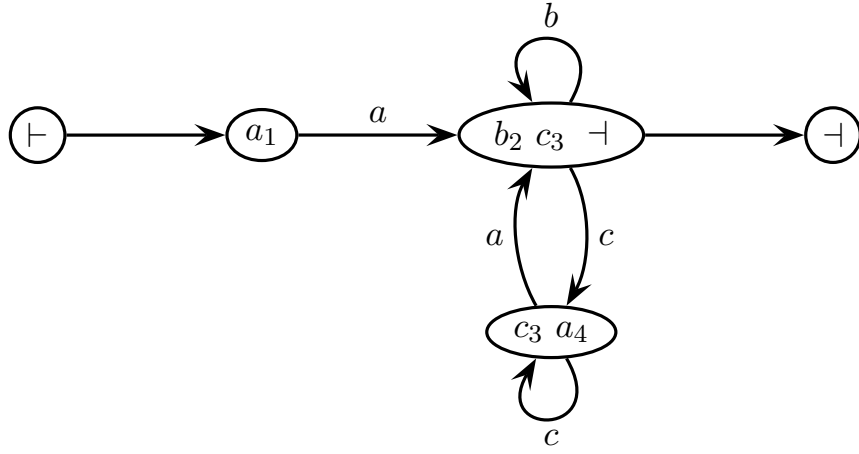
initials	a_1
terminal	followers
a_1	$b_2 \ c_3 \ \neg$
b_2	$b_2 \ c_3 \ \neg$
c_3	$c_3 \ a_4$
a_4	$b_2 \ c_3 \ \neg$



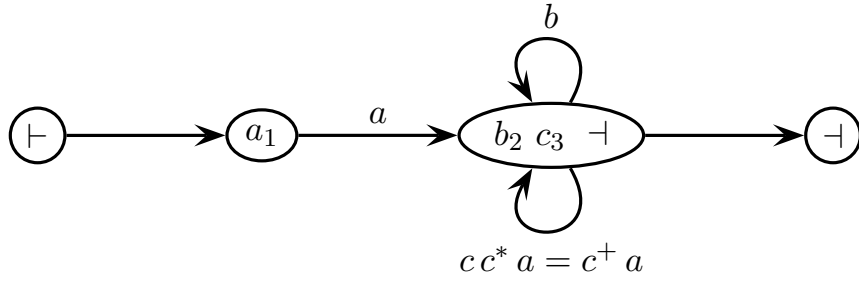
Automaton A is deterministic and is in the clean form (by construction).

(c) The (deterministic) automaton A is minimal: it is in the clean form and the two non-final states have different arcs.

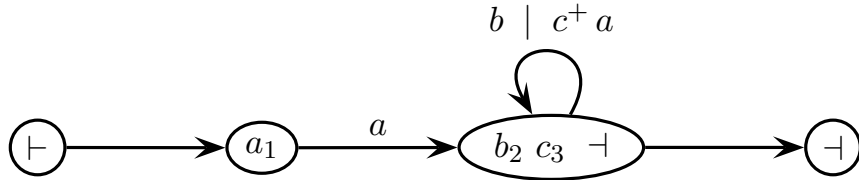
(d) Brzozowski (node elimination):



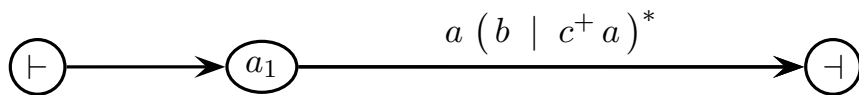
Step 1 – eliminate node $(c_3 \ a_4)$:



that is:



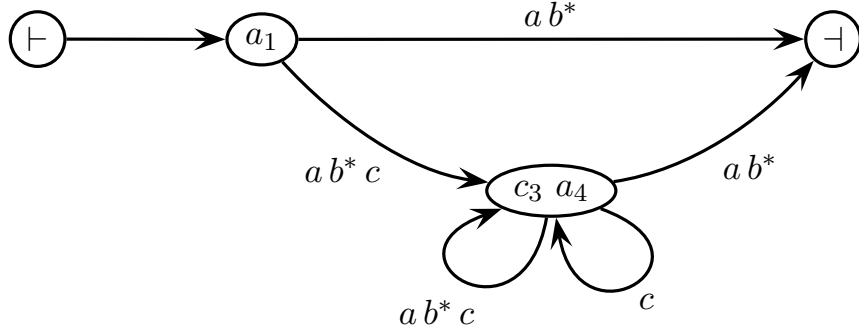
Step 2 – eliminate node $(b_2 \ c_3 \ \dashv)$:



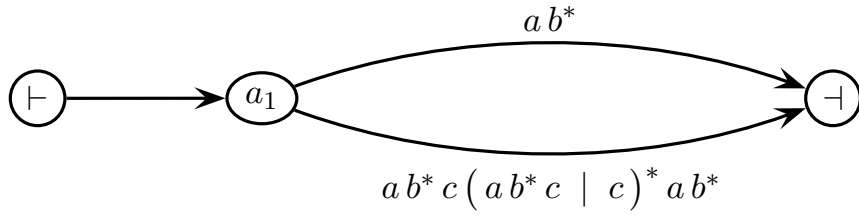
Thus we obtain:

$$R' = a (b \mid c^+ a)^*$$

A different elimination order may yield a different final regular expression. For instance, by first eliminating node $(b_2 \ c_3 \ \dashv)$ we have:



and then by eliminating node $(c_3 \ a_4)$ we have:

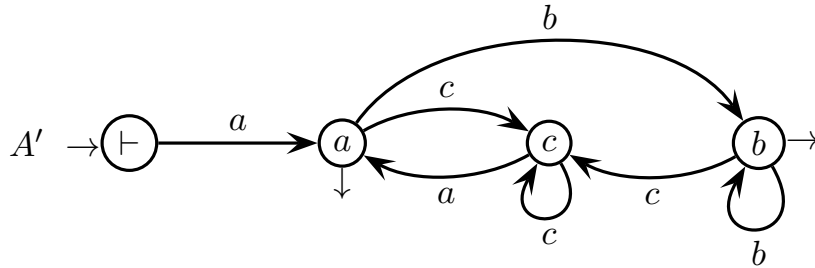


therefore a possible alternative formulation for expression R' is:

$$R' = ab^* \mid ab^*c(ab^*c \mid c)^*ab^* = ab^* [c([ab^*]c)^*ab^*]$$

which is more complicated than the former one. The reader may wish to prove that it is equivalent to expression R as well.

- (e) Language $L(R)$ is locally testable. First notice however, that the regular expression R is not linear, as letter a occurs twice, therefore the sufficient condition for local testability does not apply. We have to consider different approaches. A formal proof consists of finding the local sets of expressions R , i.e., $Ini = \{ a \}$, $Fin = \{ a, b \}$ and $Dig = \{ ab, ac, bb, bc, ca, cc \}$, then of drawing the local (normalized) automaton A' associated to such sets:



and of observing that states a and b are equivalent (both are final and have their equally labeled arcs directed to the same states). After merging such states, automata A' and A are identical, therefore their language is locally testable.

A different, still formal proof for the local testability of the language consists of observing that the deterministic automaton A is local (though not normalized), because each input letter enters exactly one state (state $(b_2 \ c_3 \ \dashv)$ is entered by two letters, namely a and b , which means that the automaton is not normalized – see the textbook), and because there is one non-reentrant initial state.

2 Free Grammars and Pushdown Automata 20%

1. Consider the language L_1 below, over the three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$L_1 = \{ a^m b c^n \mid m > n > 0 \} \cup \{ c^n b a^m \mid m > n > 0 \}$$

Examples and counterexamples:

$$a^3 b c^2, c b a^2 \in L_1$$

$$a^2 b c^3, c^3 b a^2 \notin L_1$$

Answer the following questions:

- (a) Write a *BNF* grammar G_1 , *not extended*, that generates language L_1 .
- (b) Say if your grammar G_1 is ambiguous, and suitably motivate your answer.
- (c) Draw the syntax tree, or all the trees in case of ambiguity, for the string below:

$$c^2 b a^3 \in L_1$$

Next consider the language L_2 , over the same alphabet Σ , the strings of which consist of an even number $k \geq 4$ of non-empty groups of letters a and of letters c . The groups alternate and start with letters a , such that:

- the groups of letters a and the groups of letters c are separated by $k - 1$ isolated letters b
- the groups of letters a and the groups of letters c that are located in a symmetric position with respect to the central letter b , have the same length

Examples and counterexamples:

$$a^3 b c^2 b a^2 b c^3, a b c b a b c b a b c \in L_2$$

$$a^3 b c^2 b a^3, a^3 b c^2 b a^3 b c^2 \notin L_2$$

Answer the following questions:

- (d) Write a *BNF* grammar G_2 , *not extended and not ambiguous*, that generates language L_2 .
- (e) Draw the syntax trees for the two strings below:

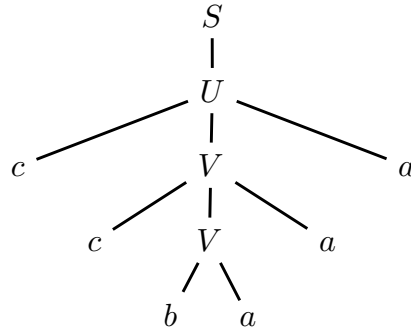
$$a b c^2 b a^2 b c, a b c b a b c b a b c \in L_2$$

Solution

- (a) Here is a *BNF* grammar G_1 for language L_1 (axiom S):

$$G_1 \left\{ \begin{array}{l} S \rightarrow X \mid U \\ X \rightarrow a Y c \\ Y \rightarrow a Y \mid a Y c \mid a b \\ U \rightarrow c V a \\ V \rightarrow V a \mid c V a \mid b a \end{array} \right.$$

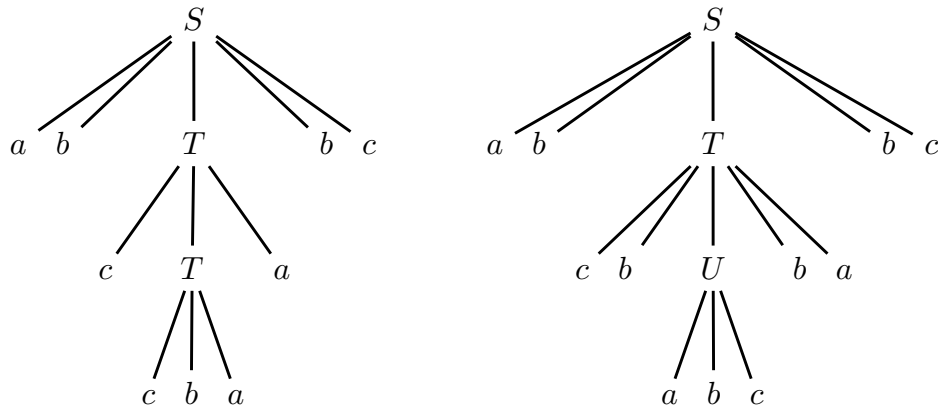
- (b) (*sketchy*) Grammar G_1 is ambiguous. For instance, in a derivation the alternatives $a Y$ and $a Y c$ in the rule $Y \rightarrow a Y \mid a Y c \mid a b$ may be taken in any order and correspond to different trees, though resulting in the same string.
- (c) The specific string $c^2 b a^3 \in L_1$ is not ambiguous, anyway. There is only one syntax tree, below:



- (d) Here is a *BNF* grammar G_2 for language L_2 (axiom S):

$$G_2 \left\{ \begin{array}{l} S \rightarrow a S c \mid a b T b c \\ T \rightarrow c T a \mid c b U b a \mid c b a \\ U \rightarrow a U c \mid a b T b c \mid a b c \end{array} \right.$$

- (e) Here are the syntax trees for the two strings abc^2ba^2bc , $abcbabcbabcb \in L_2$:



2. We intend to model the *Boolean* expressions of an Algol-like programming language. Such expressions feature the operators below, listed in order of decreasing precedence:

- $<$ and $>$: relational operators, binary, applicable only to arithmetic expressions
- **not**: negation, unary
- **and**: conjunction, binary, right-associative
- **or**: disjunction, binary, right-associative
- **impl**: implication, binary, right-associative
- **if ... then ... else ... fi**: conditional

All the above logical operators (except for the relational operators) are applicable only to Boolean expressions.

Examples:

$v1 > z3$	– relational operator
$x > z$ or $z < y$	– logical and relational operators
r and (s impl $x > y$)	– parentheses are allowed
if if not r then q else f fi then s else t fi	– structure if can be nested

Answer the following questions:

- (a) Write a grammar, *BNF*, *not extended and not ambiguous*, that defines the Boolean expression language outlined above. Boolean atomic expressions (constants and variables) are represented by the terminal symbol **b**. Arithmetic expressions are represented by the terminal symbol **a**.
- (b) Draw the syntax tree for the Boolean expression below:

r and (s impl $x > y$)

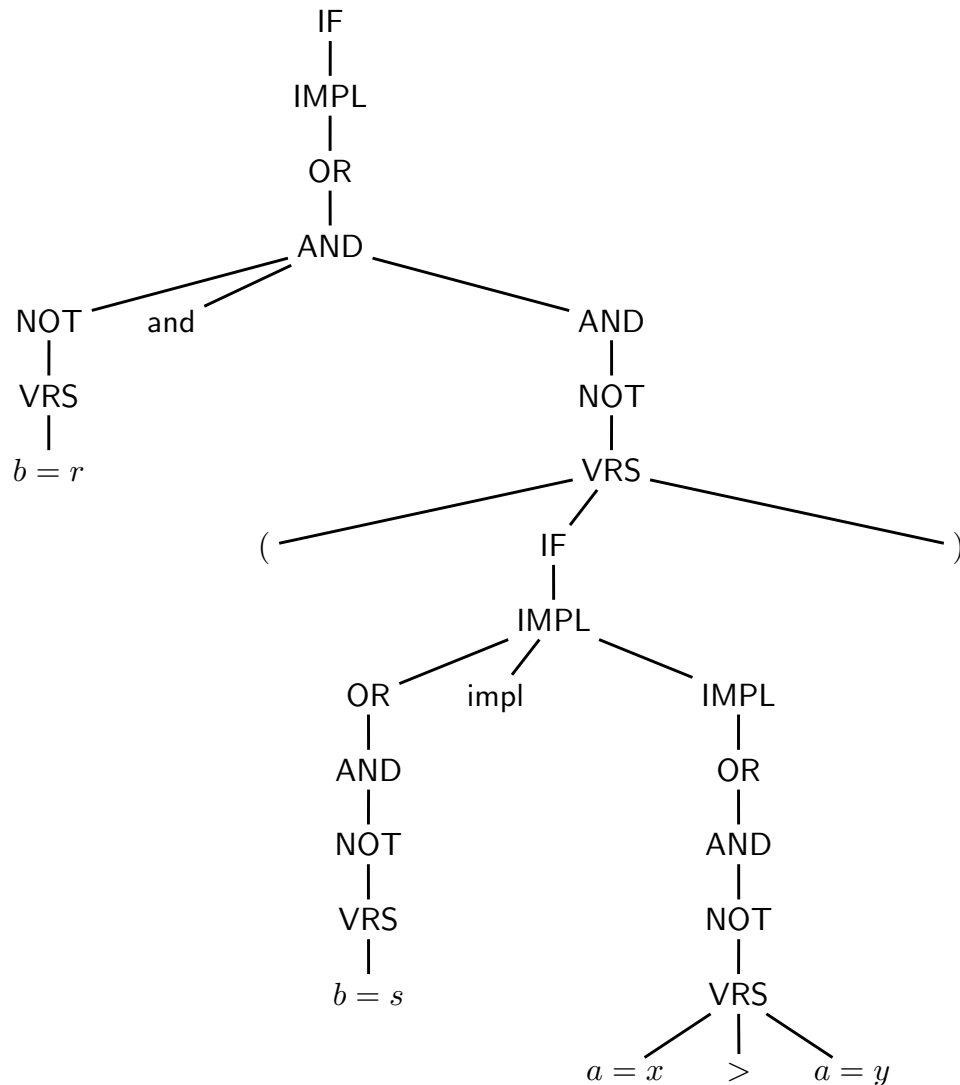
Solution

(a) Here below is the grammar (axiom $\langle IF \rangle$), of type *BNF*:

$$\left\{ \begin{array}{l} \langle IF \rangle \rightarrow \text{if } \langle IF \rangle \text{ then } \langle IF \rangle \text{ else } \langle IF \rangle \text{ fi} \mid \langle IMPL \rangle \\ \langle IMPL \rangle \rightarrow \langle OR \rangle \text{ impl } \langle IMPL \rangle \mid \langle OR \rangle \\ \langle OR \rangle \rightarrow \langle AND \rangle \text{ or } \langle OR \rangle \mid \langle AND \rangle \\ \langle AND \rangle \rightarrow \langle NOT \rangle \text{ and } \langle AND \rangle \mid \langle NOT \rangle \\ \langle NOT \rangle \rightarrow \text{not } \langle VRS \rangle \mid \langle VRS \rangle \\ \langle VRS \rangle \rightarrow \underbrace{b}_{\text{Variable}} \mid \underbrace{a < a}_{\text{Relational}} \mid \underbrace{a > a}_{\text{Relational}} \mid \underbrace{(\langle IF \rangle)}_{\text{Subexpr.}} \end{array} \right.$$

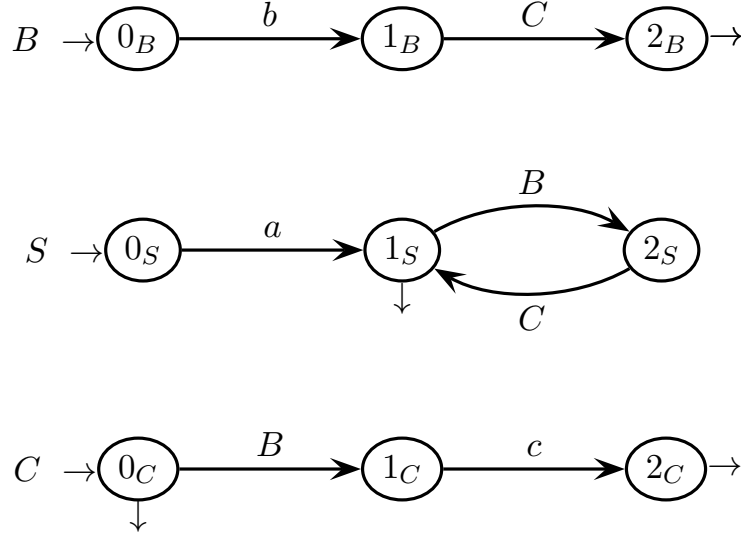
The grammar is based on right-recursive rules to model the right associativity of the operators, produces a non-ambiguous if-fi, and does not have any circular derivations. All this reasonably grants that the grammar is not ambiguous.

(b) Here below is the syntax tree for expression “ r and (s impl $x > y$)”:



3 Syntax Analysis and Parsing Methodologies 20%

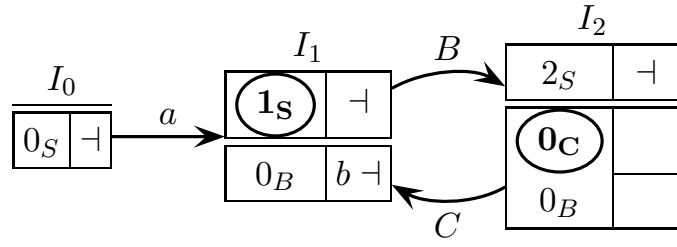
1. Consider the *EBNF* grammar G below, presented as a net of machines, over the nonterminal and terminal alphabets $V = \{ B, C, S \}$ and $\Sigma = \{ a, b, c \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

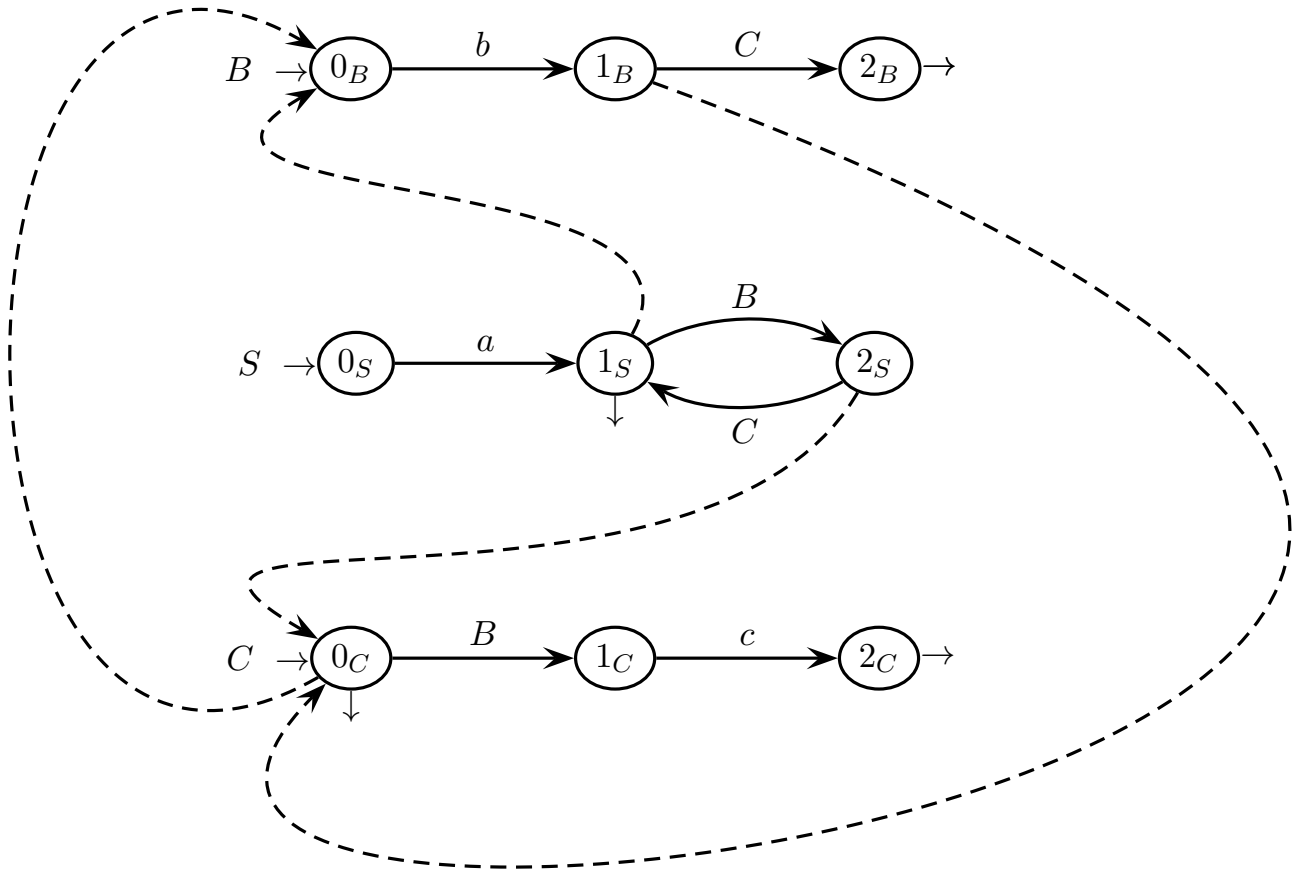
- (a) Complete the graph of the ELR pilot.
 - (b) Show that grammar G is not of type $ELR(1)$ and list all the pilot conflicts.
 - (c) Complete the machine net with the guide sets on the call arcs and exit arrows.
 - (d) Show that grammar G is not of type $ELL(1)$ and list all the guide set conflicts.
 - (e) Is the language $L(G)$ regular ? Justify your answer.
 - (f) (optional) As machine M_B does not have any bifurcation points, such a machine is of type $ELL(1)$ independently of the other machines. Write the recursive descent syntactic procedure of machine M_B .
-

question (a) – pilot graph to be completed



question (b) – list of pilot conflicts

question (c) – machine net to be completed with guide sets



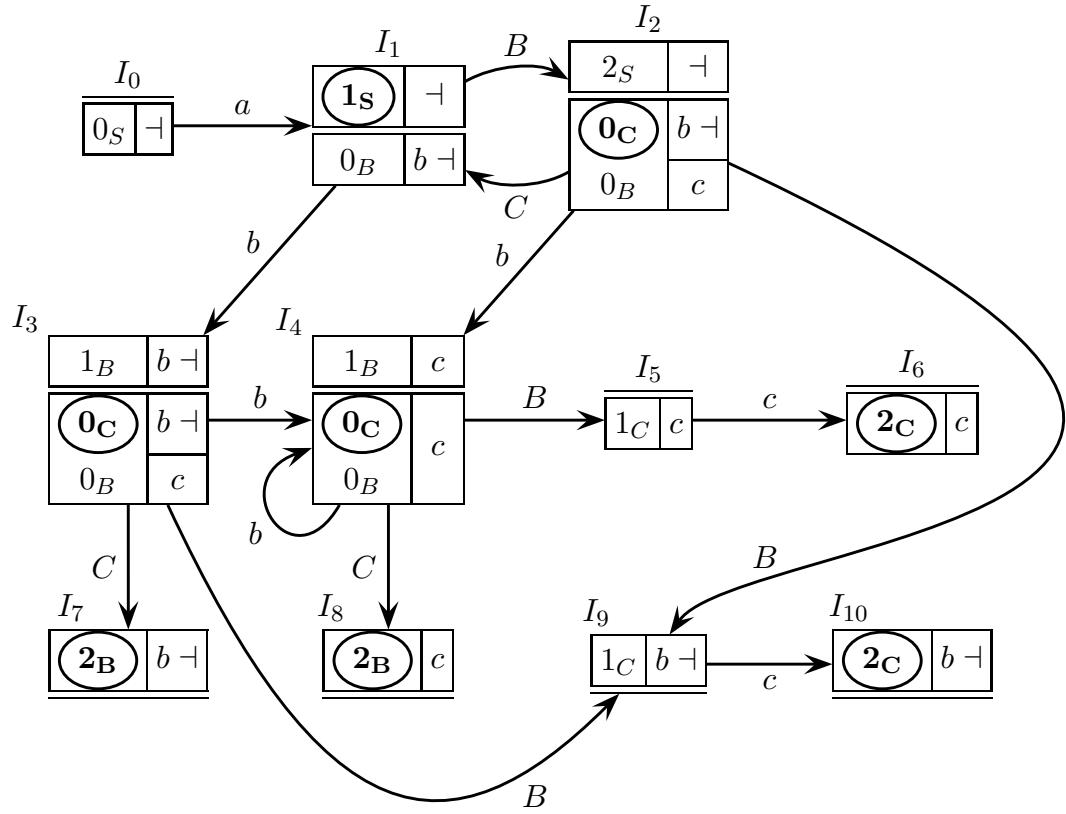
question (d) – list of guide set conflicts

question (e) – nature of language $L(G)$

question (f) – syntactic procedure of machine M_B

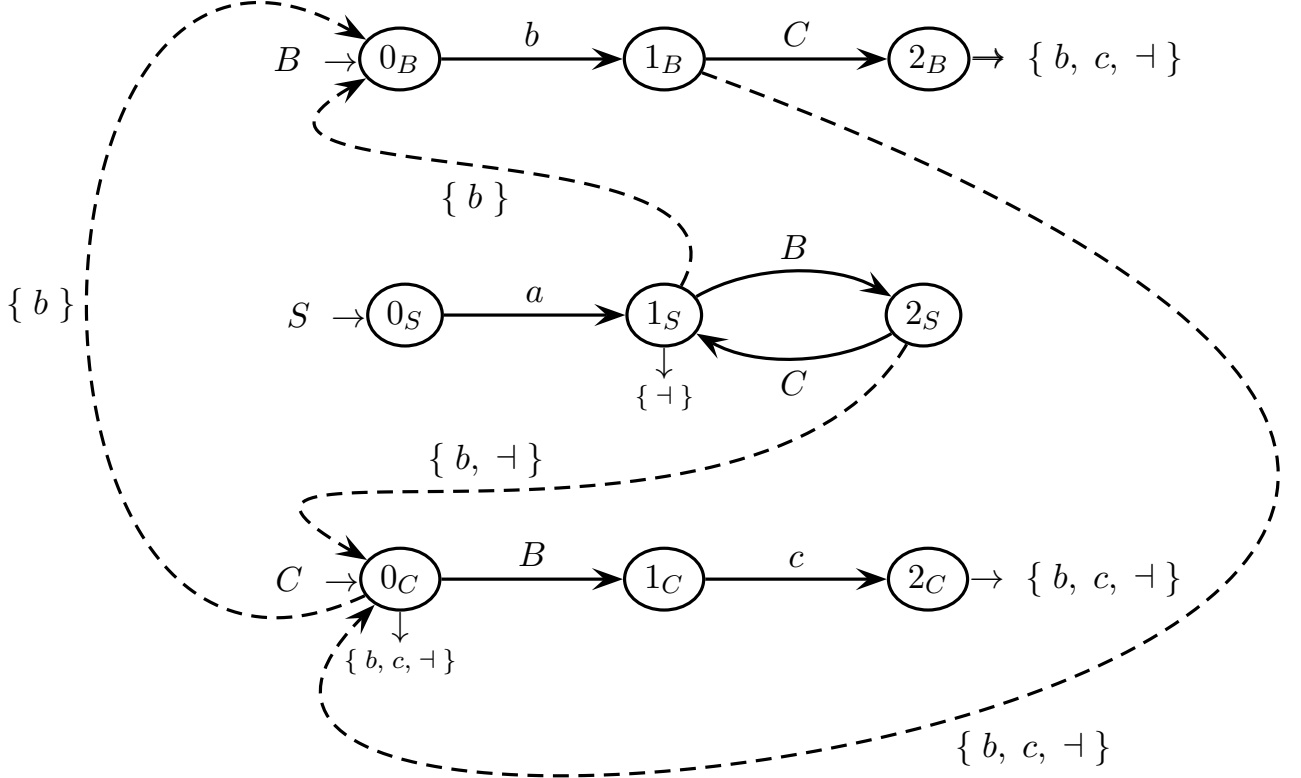
Solution

(a) Pilot with eleven m-states:



(b) Two SR conflicts in I_2 and I_3 , both for the final item 0_C on the look-ahead b .

(c) Machine net with guide sets (PCFG):



- (d) Only one conflict in the state 0_C between the call arc to machine M_B and the exit arrow, on terminal b .
- (e) Language $L(G)$ is not regular, since grammar G has a self-embedding (autoinclusive) derivation. In fact we have:

$$L(B) = b L(C) = b (\varepsilon \mid L(B) c) = b \mid b L(B) c$$

and the recursive component $b L(B) c$ is self-embedding. This suffices to reasonably conclude that the language of grammar G is not regular.

More precisely, we have $L(B) = b^{h+1} c^h$ for any $h \geq 0$, hence $L(C) = \varepsilon \mid L(B) c = \varepsilon \mid b^{h+1} c^h c = b^k c^k$ for any $k \geq 0$, whence:

$$L(S) = a (L(B) L(C))^* = a \underbrace{b^{n_1+1} c^{n_1} b^{n_2} c^{n_2}}_{\text{1-st star iteration}} \dots \underbrace{b^{n_{2l-1}+1} c^{n_{2l-1}} b^{n_{2l}} c^{n_{2l}}}_{\text{l-th star iteration}}$$

for any $l \geq 1$ and $n_s \geq 0$ with $1 \leq s \leq 2l$, and $L(S) \ni a$ if the star nullifies. Since $L(G) = L(S)$, again clearly the language of grammar G is not regular, because its strings contain substrings that have two equal (or almost equal) exponents, a typical non-regular structure.

(f) Here is the syntactic procedure of machine M_B :

```

procedure  $B$                                      // initial state  $0_B$ 
if  $cc \in \{ b \}$  then
     $cc := next$                                      // goto state  $1_B$ 
    if  $cc \in \{ b, c, \dashv \}$  then
        call  $C$                                      // goto state  $2_B$ 
        if  $cc \in \{ b, c, \dashv \}$  then
            return                                   // exit state  $2_B$ 
        else error                                  // error in state  $2_B$ 
    else error                                      // error in state  $1_B$ 
else error                                         // error in state  $0_B$ 

```

Of course, it can be compacted by grouping the error cases:

```

procedure  $B$                                      // initial state  $0_B$ 
if  $cc \in \{ b \}$  then
     $cc := next$                                      // goto state  $1_B$ 
    if  $cc \in \{ b, c, \dashv \}$  then
        call  $C$                                      // goto state  $2_B$ 
        if  $cc \in \{ b, c, \dashv \}$  then
            return                                   // exit state  $2_B$ 
    error                                           // error somewhere in machine  $M_B$ 

```

though it loses some precision in detecting the error type.

4 Language Translation and Semantic Analysis 20%

1. The *BNF* grammar G below (axiom S):

$$G: S \rightarrow o S c S \mid \varepsilon$$

defines the Dyck language with brackets “ o ” (open) and “ c ” (closed). We intend to define a translation scheme τ , where the bracket pairs placed at an **odd** nesting level are turned into pairs “ a ” and “ b ”, while those placed at an **even** nesting level are turned into pairs “ d ” and “ e ”. Notice that the outermost brackets are considered at level 1 (see also the example).

Here is a sample translation (spacing visually helps identifying the bracket pairs):

$$\begin{aligned} s &= o o c c o o c o o c c c \\ \tau(s) &= a d e b a d e d a b e b \end{aligned}$$

Answer the following questions:

- (a) Modify the above grammar G so as to obtain the *BNF* source grammar G_s of a translation scheme suitable for the definition of the translation τ specified above.
- (b) Complete the translation scheme with the *BNF* target (destination) grammar G_t that matches the source grammar G_s defined for the previous question.
- (c) Draw the source and target (destination) syntax trees for the source string:

$$s = o c o o o c c c$$

Solution

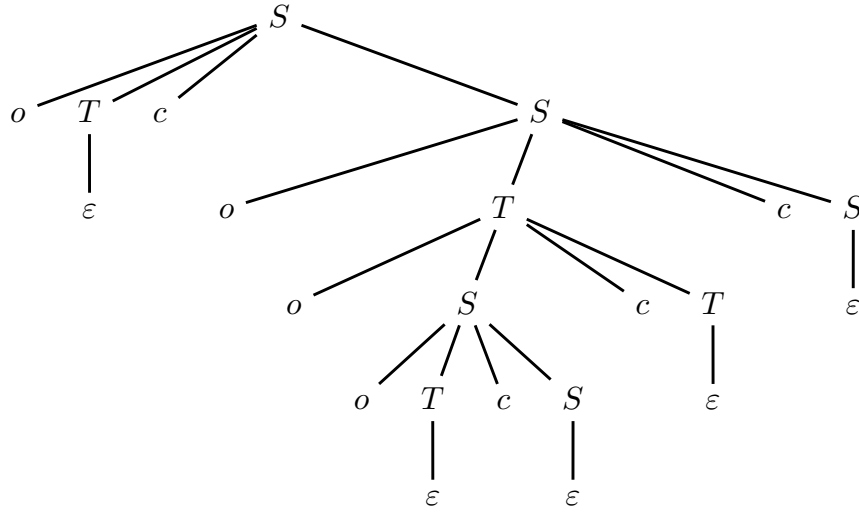
(a) Here is a source grammar G_s (axiom S):

$$G_s \begin{cases} S \rightarrow o T c S \mid \varepsilon \\ T \rightarrow o S c T \mid \varepsilon \end{cases}$$

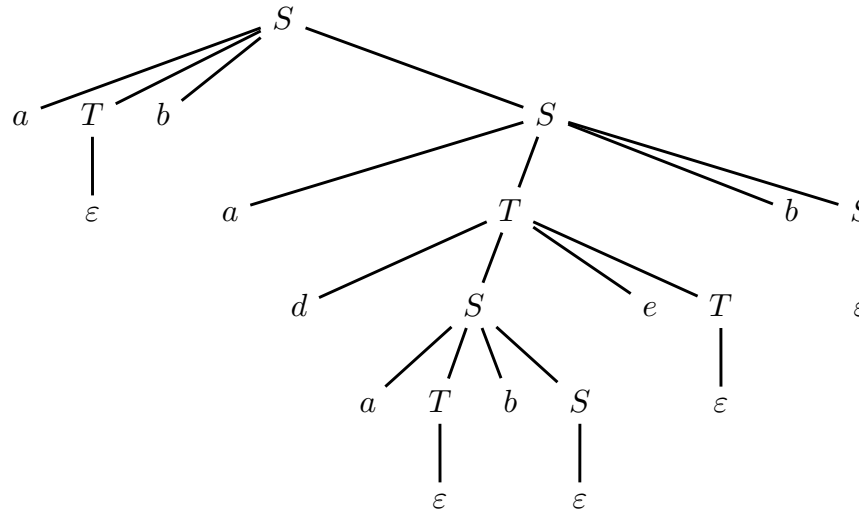
(b) Here is the target (destination) grammar G_t (axiom S):

$$G_t \begin{cases} S \rightarrow a T b S \mid \varepsilon \\ T \rightarrow d S e T \mid \varepsilon \end{cases}$$

(c) Source tree for the string $s = o c o o o c c c$:



Target (destination) tree for the string $\tau(s) = \tau(o c o o o c c c) = a b a d a b e b$:



2. We intend to translate an arithmetic expression from the infix form to the postfix or prefix forms, by using an attribute grammar instead of a syntactic translation scheme. We refer to the following simplified grammar G (axiom S) as syntactic support. Grammar G features only right associative binary addition “+”, round brackets “(” and “)”, and atomic expressions (constants and variables) represented by terminal symbol a :

$$G \left\{ \begin{array}{l} S \rightarrow E \\ E \rightarrow T + E \\ E \rightarrow T \\ T \rightarrow (E) \\ T \rightarrow a \end{array} \right.$$

A first translation τ_1 simply produces the usual postfix form and eliminates the brackets. For instance:

$$\tau_1 (a +_1 (a +_2 a) +_3 a) = a a a +_2 a +_3 +_1$$

where the subscripts are added to the operator “+” only for the sake of readability.

Answer the following questions (use the figures / tables / spaces on the next pages):

- Write an attribute grammar, with grammar G as syntactic support, that computes translation τ_1 (without subscripts on the operator “+”) by using only a string *left* attribute s associated with the nonterminals S , E and T . Translation τ_1 is computed as the attribute s of the root node S .
- Decorate the tree (see the next page) of the source string “ $a +_1 (a +_2 a) +_3 a$ ” by applying the values of attribute s at each nonterminal node.

A second translation τ_2 keeps the round brackets, produces the postfix form for the (sub)expressions that are enclosed in an *even* number (including zero) of bracket pairs, and produces the *prefix* form for the (sub)expressions that are enclosed in an *odd* number of bracket pairs. For instance:

$$\tau_2 (a +_1 (a +_2 a) +_3 a) = a (+_2 a a) a +_3 +_1$$

$$\tau_2 (a +_1 (a +_2 ((a +_3 a) +_4 a)) +_5 a) = a(+_2 a((+_3 a a) a +_4)) a +_5 +_1$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- Write an attribute grammar, with grammar G as syntactic support, that computes translation τ_2 (without subscripts on the operator “+”) by using two attributes: a boolean *right* attribute e associated with nonterminals E and T , and a string *left* attribute s associated with nonterminals S , E and T . Attribute e means that the current (sub)expression is enclosed in an even number of bracket pairs. Translation τ_2 is computed as the attribute s of the root node S .
- Decorate the tree (in the second next page) of the source string “ $a +_1 (a +_2 a) +_3 a$ ” by applying the values of attributes e and s at each nonterminal node.

syntax

semantics for translation τ_1 – question (a)

$$1: S_0 \rightarrow E_1$$

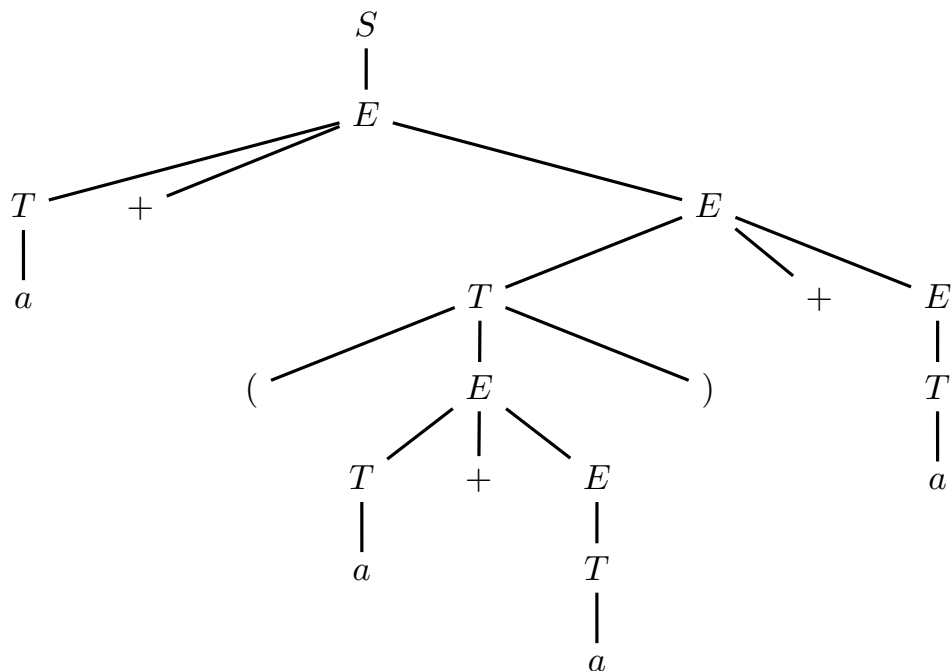
$$2: E_0 \rightarrow T_1 + E_2$$

$$3: E_0 \rightarrow T_1$$

$$4: T_0 \rightarrow (E_1)$$

$$5: T_0 \rightarrow a$$

question (b) – tree to be decorated



syntax

semantics for translation τ_2 – question (c)

1: $S_0 \rightarrow E_1$

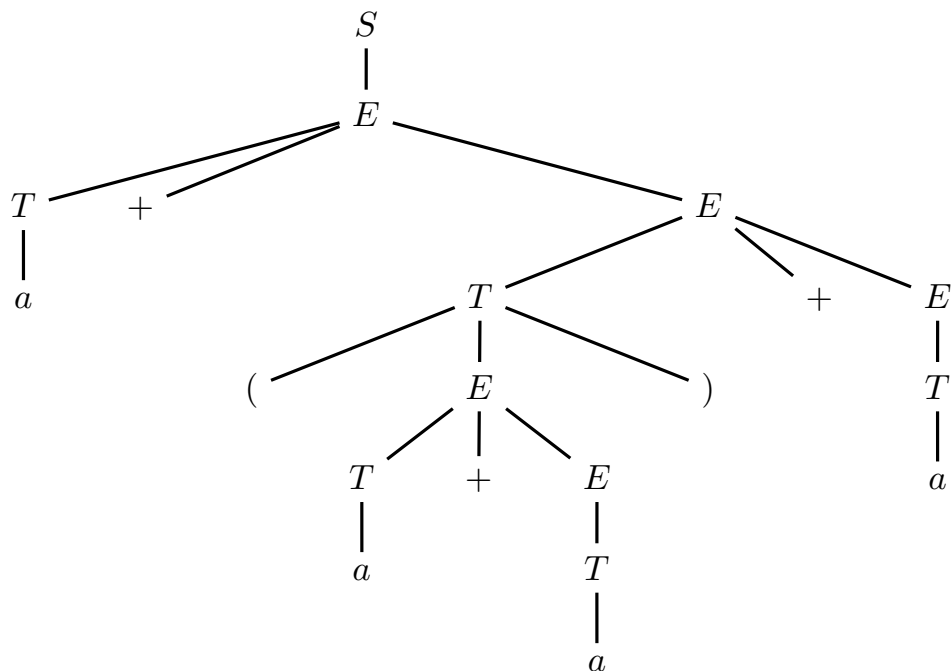
2: $E_0 \rightarrow T_1 + E_2$

3: $E_0 \rightarrow T_1$

4: $T_0 \rightarrow (E_1)$

5: $T_0 \rightarrow a$

question (d) – tree to be decorated

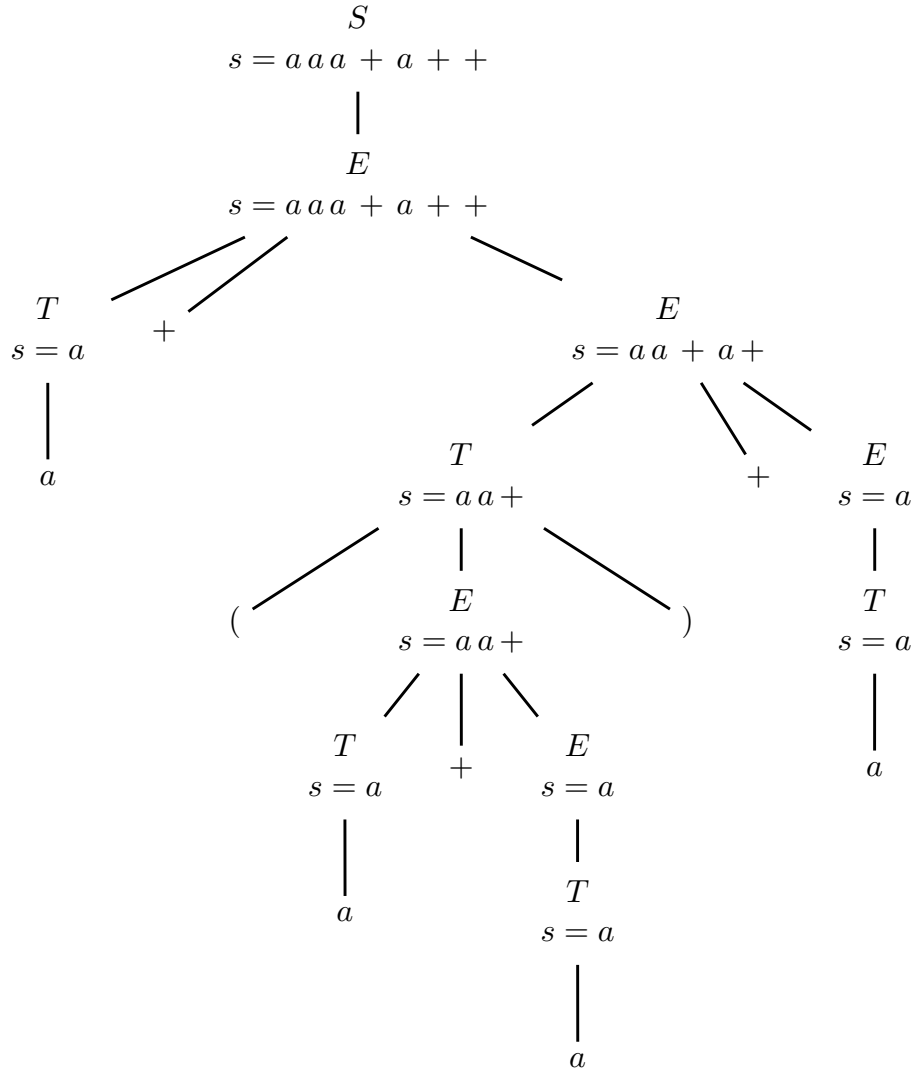


Solution

(a) Here is an attribute grammar for translation τ_1 :

#	syntax	semantics for translation τ_1 – question (a)
1:	$S_0 \rightarrow E_1$	$s_0 := s_1$
2:	$E_0 \rightarrow T_1 + E_2$	$s_0 := s_1 \cdot s_2 \cdot +$
3:	$E_0 \rightarrow T_1$	$s_0 := s_1$
4:	$T_0 \rightarrow (E_1)$	$s_0 := s_1$
5:	$T_0 \rightarrow a$	$s_0 := a$

(b) Here is the syntax tree of string “ $a +_1 (a +_2 a) +_3 a$ ”, with the values of attribute s at each nonterminal node.



(c) Here is an attribute grammar for translation τ_2 :

#	<i>syntax</i>	<i>semantics for translation τ_2 – question (c)</i>
1:	$S_0 \rightarrow E_1$	$e_1 := true$ $s_0 := s_1$
2:	$E_0 \rightarrow T_1 + E_2$	$e_1, e_2 := e_0$ if e_0 then $s_0 := s_1 \cdot s_2 \cdot +$ else $s_0 := + \cdot s_1 \cdot s_2$
3:	$E_0 \rightarrow T_1$	$e_1 := e_0$ $s_0 := s_1$
4:	$T_0 \rightarrow (E_1)$	$e_1 := \mathbf{not} \ e_0$ $s_0 := (\cdot s_1 \cdot)$
5:	$T_0 \rightarrow a$	$s_0 := a$

(d) Here is the syntax tree of string “ $a+_1(a+_2a)+_3a$ ”, with the values of attributes e and s at each nonterminal node (symbols T and F denote *true* and *false*).

